

SGT UNIVERSITY
School of Engineering and Technology
Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 3

SQL Joins — CROSS, NATURAL, INNER, OUTER & SELF

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	02 September 2026
Dataset	departments (6) + employees (7) · deptID + managerID
Methods	CROSS JOIN, NATURAL JOIN, INNER JOIN, LEFT/RIGHT OUTER JOIN, SELF JOIN

1. Aim

To combine rows from related tables using `CROSS JOIN`, `NATURAL JOIN`, `INNER JOIN`, `LEFT OUTER JOIN`, `RIGHT OUTER JOIN`, and `SELF JOIN`; to compare how each join type treats unmatched rows; and to understand when to use each one.

2. Theory

2.1 CROSS JOIN

A `CROSS JOIN` returns the Cartesian product of two tables: every row from the first table is paired with every row from the second. There is no join condition. With 6 departments and 7 employees the result has $6 \times 7 = 42$ **rows**. Cross joins are rare in everyday queries but useful for generating combinations (e.g. all size–colour pairs) or as a building block for other joins.

2.2 NATURAL JOIN

A `NATURAL JOIN` automatically matches rows on every column that appears in both tables with the same name. Here, both tables share `deptID`, so the condition is implicit. Only rows with a matching `deptID` in both tables are returned — the same six rows as `INNER JOIN` when `deptID` is the sole common column. (SQL Server does not support `NATURAL JOIN`; use an explicit `INNER JOIN` instead.)

2.3 INNER JOIN

An `INNER JOIN` returns only rows where the explicit join condition is satisfied. Unmatched rows from either table are discarded. It is the most common join: use it when you only want records that have a partner on both sides.

2.4 LEFT OUTER JOIN

A `LEFT OUTER JOIN` keeps every row from the left table. If no match exists in the right table, the right-hand columns are filled with `NULL`. Employee Neha Gupta (`deptID` 99) has no matching department and therefore appears with a null department name.

2.5 RIGHT OUTER JOIN

A `RIGHT OUTER JOIN` keeps every row from the right table. Unmatched left-hand columns become `NULL`. School of Humanities (`deptID` 6) has no employees and therefore appears with null employee fields. A `RIGHT JOIN` can always be rewritten as a `LEFT JOIN` by swapping the table order.

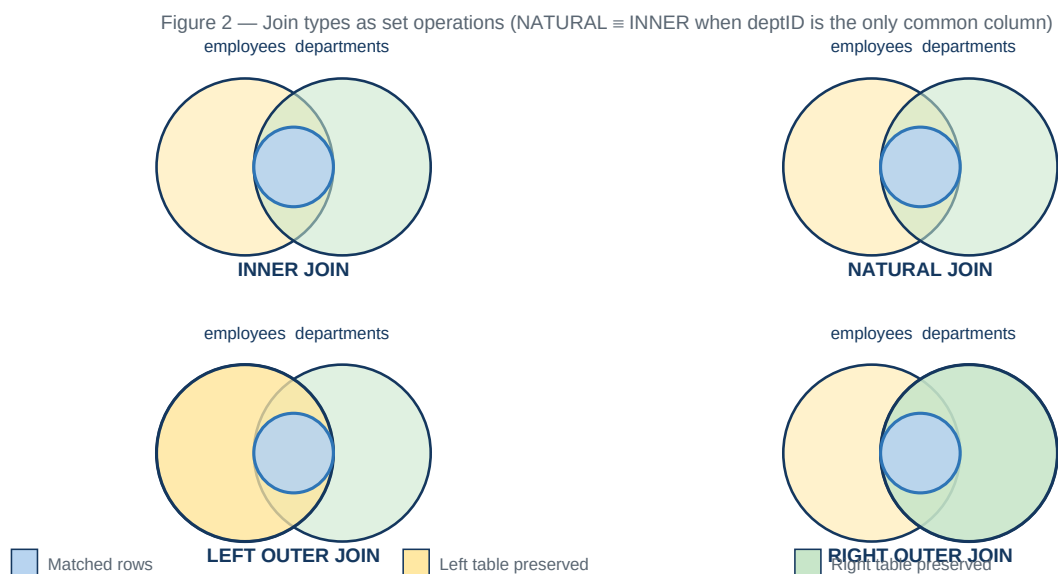
2.6 SELF JOIN

A `SELF JOIN` joins a table to itself using two different aliases. It is not a separate SQL keyword — it is an `INNER` or `LEFT JOIN` where both sides are the same table. Here, `employees e` is joined to `employees m` on `e.managerID = m.empID` to list each employee alongside their manager's name.

2.7 Differences between all join types

Join type	Join condition?	Unmatched rows	Typical use
CROSS JOIN	None (Cartesian product)	N/A — all combinations	Generate all pairs
NATURAL JOIN	Implicit (same column names)	Dropped from both sides	Quick join on shared key
INNER JOIN	Explicit ON clause	Dropped from both sides	Only matching records
LEFT OUTER JOIN	Explicit ON clause	Left kept; right NULL	Keep all from left table
RIGHT OUTER JOIN	Explicit ON clause	Right kept; left NULL	Keep all from right table
SELF JOIN	Explicit ON (same table)	Depends on INNER/LEFT used	Hierarchies, comparisons

Key distinctions: CROSS JOIN multiplies rows with no filter. INNER and NATURAL return only matches. LEFT and RIGHT OUTER preserve one side's orphans. SELF JOIN is a technique (same table, two aliases), not a separate result category — it uses inner or outer join semantics on a single table.



3. Schema

Two tables are linked by deptID. employees also references itself through managerID for the self join.

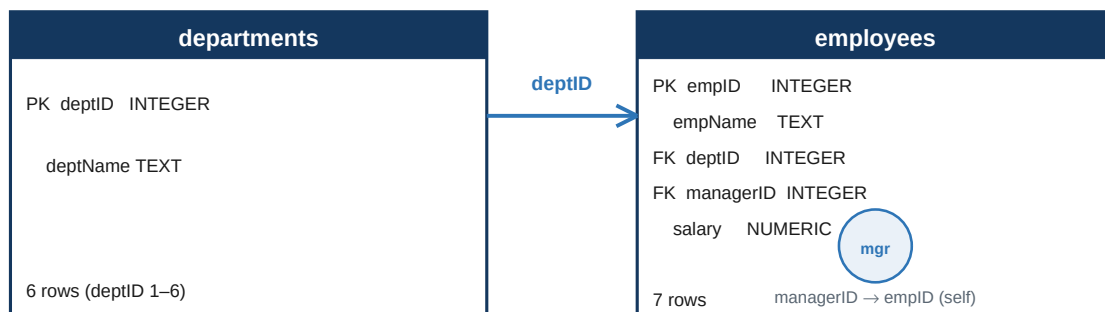


Figure 1 — Schema: employees.deptID references departments.deptID

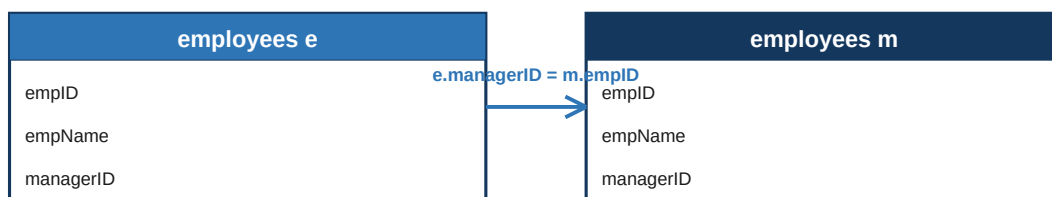


Figure 5 — SELF JOIN: same table, two aliases

Table	Attribute	Type	Role
departments	deptID	INTEGER / INT	Primary key
departments	deptName	TEXT / VARCHAR	School name
employees	empID	INTEGER / INT	Primary key
employees	empName	TEXT / VARCHAR	Employee name
employees	deptID	INTEGER / INT	FK → departments
employees	managerID	INTEGER / INT	FK → employees (self)
employees	salary	NUMERIC(10,2)	Monthly salary

4. Procedure

1. Create departments and employees (with managerID for self join).
2. Insert six departments and seven employees, including deliberate mismatches for outer joins.
3. Display both base tables.
4. Run `CROSS JOIN` — expect 42 rows (count shown; sample limited to 8).
5. Run `NATURAL JOIN` and `INNER JOIN` — expect six matched rows.
6. Run `LEFT OUTER JOIN` — expect seven rows (one employee without a department).
7. Run `RIGHT OUTER JOIN` — expect seven rows (one department without employees).
8. Run `SELF JOIN` — expect seven rows listing each employee and their manager.

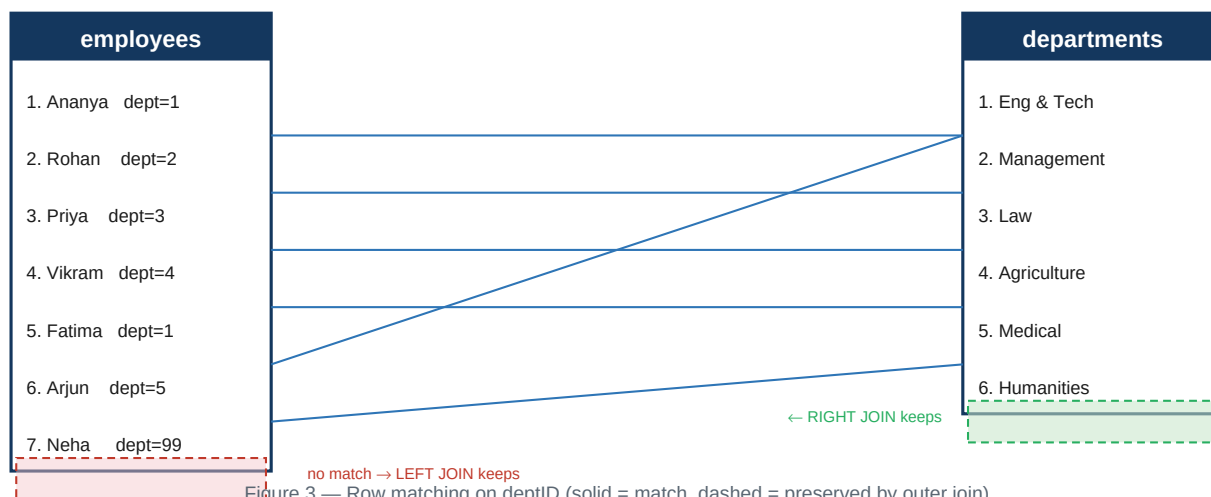


Figure 3 — Row matching on deptID (solid = match, dashed = preserved by outer join)

5. Source Code

```
-- =====
-- DBMS Lab · 02-09-2026
-- SQL Joins: CROSS, NATURAL, INNER, OUTER (LEFT/RIGHT), SELF
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run joins.sqlserver.sql instead.
-- =====

DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;

-- -----
-- 1. Create related tables
--   • deptID links employees → departments
--   • managerID links employees → employees (for SELF JOIN)
-- -----
CREATE TABLE departments (
    deptID    INTEGER PRIMARY KEY,
    deptName  TEXT NOT NULL
);

CREATE TABLE employees (
    empID      INTEGER PRIMARY KEY,
    empName    TEXT NOT NULL,
    deptID     INTEGER NOT NULL,
    managerID  INTEGER,
    salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);

-- -----
-- 2. Insert sample rows
--   • deptID 6 (Humanities) has no employees → RIGHT OUTER JOIN
--   • empID 7 points to deptID 99 (missing) → LEFT OUTER JOIN
--   • managerID links form a simple reporting chain → SELF JOIN
-- -----
INSERT INTO departments (deptID, deptName) VALUES
    (1, 'School of Engineering and Technology'),
    (2, 'School of Management'),
    (3, 'School of Law'),
    (4, 'School of Agricultural Sciences'),
    (5, 'School of Medical Sciences'),
    (6, 'School of Humanities');

INSERT INTO employees (empID, empName, deptID, managerID, salary) VALUES
    (1, 'Ananya Sharma', 1, NULL, 92000.00),
    (2, 'Rohan Mehta', 2, 1, 78000.00),
    (3, 'Priya Nair', 3, 1, 85000.00),
    (4, 'Vikram Singh', 4, 2, 76000.00),
    (5, 'Fatima Khan', 1, 1, 88000.00),
    (6, 'Arjun Patel', 5, 3, 95000.00),
    (7, 'Neha Gupta', 99, 2, 54000.00);

-- -----
-- 3. Base tables (reference)
-- -----
SELECT * FROM departments;
SELECT * FROM employees;

-- -----
-- 4. CROSS JOIN
--   Cartesian product: every row of table A paired with every
--   row of table B. No join condition. Here: 6 × 7 = 42 rows.
-- -----
SELECT COUNT(*) AS cross_join_row_count
FROM departments
CROSS JOIN employees;

SELECT d.deptName, e.empName
FROM departments AS d
CROSS JOIN employees AS e
ORDER BY d.deptID, e.empID
LIMIT 8;

-- -----
-- 5. NATURAL JOIN
--   Matches on every column with the same name (here: deptID).
--   Only rows with a matching deptID in BOTH tables appear.
-- -----
SELECT empID, empName, deptName, salary
FROM employees
NATURAL JOIN departments;

-- -----
-- 6. INNER JOIN
--   Explicit join condition; same result as NATURAL JOIN here
```

```

-- because deptID is the only common column.
-----
SELECT e.empID, e.empName, d.deptName, e.salary
FROM employees AS e
INNER JOIN departments AS d ON e.deptID = d.deptID;

-- -----
-- 7a. LEFT OUTER JOIN
-- All rows from the LEFT table (employees); unmatched dept
-- columns are NULL (empID 7 → deptID 99 has no department).
-----
SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
FROM employees AS e
LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;

-- -----
-- 7b. RIGHT OUTER JOIN
-- All rows from the RIGHT table (departments); unmatched
-- employee columns are NULL (deptID 6 has no employees).
-- Requires SQLite 3.39+; see joins.sqlserver.sql for T-SQL.
-----
SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
FROM employees AS e
RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;

-- -----
-- 8. SELF JOIN
-- A table joined to itself using different aliases.
-- Here: each employee matched to their manager (also in
-- employees). Ananya has no manager → manager name is NULL.
-----
SELECT
    e.empName AS employee,
    m.empName AS manager
FROM employees AS e
LEFT JOIN employees AS m ON e.managerID = m.empID
ORDER BY e.empID;

```

6. Output

The script was executed in SQLite. Each query's result set is shown below in execution order.

```

SQL> DROP TABLE IF EXISTS employees;
-- OK

SQL> DROP TABLE IF EXISTS departments;
-- OK

SQL> CREATE TABLE departments (
    deptID INTEGER PRIMARY KEY,
    deptName TEXT NOT NULL
);
-- OK

SQL> CREATE TABLE employees (
    empID INTEGER PRIMARY KEY,
    empName TEXT NOT NULL,
    deptID INTEGER NOT NULL,
    managerID INTEGER,
    salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);
-- OK

SQL> INSERT INTO departments (deptID, deptName) VALUES
    (1, 'School of Engineering and Technology'),
    (2, 'School of Management'),
    (3, 'School of Law'),
    (4, 'School of Agricultural Sciences'),
    (5, 'School of Medical Sciences'),
    (6, 'School of Humanities');
-- 6 row(s) inserted

SQL> INSERT INTO employees (empID, empName, deptID, managerID, salary) VALUES
    (1, 'Ananya Sharma', 1, NULL, 92000.00),
    (2, 'Rohan Mehta', 2, 1, 78000.00),
    (3, 'Priya Nair', 3, 1, 85000.00),
    (4, 'Vikram Singh', 4, 2, 76000.00),
    (5, 'Fatima Khan', 1, 1, 88000.00),
    (6, 'Arjun Patel', 5, 3, 95000.00),
    (7, 'Neha Gupta', 99, 2, 54000.00);
-- 13 row(s) inserted

SQL> SELECT * FROM departments;
deptID deptName
-----
1 School of Engineering and Technology
2 School of Management

```

```

3      School of Law
4      School of Agricultural Sciences
5      School of Medical Sciences
6      School of Humanities

```

```
SQL> SELECT * FROM employees;
```

empID	empName	deptID	managerID	salary
1	Ananya Sharma	1		92000
2	Rohan Mehta	2	1	78000
3	Priya Nair	3	1	85000
4	Vikram Singh	4	2	76000
5	Fatima Khan	1	1	88000
6	Arjun Patel	5	3	95000
7	Neha Gupta	99	2	54000

```
SQL> SELECT COUNT(*) AS cross_join_row_count
FROM departments
CROSS JOIN employees;
cross_join_row_count
-----
42
```

```
SQL> SELECT d.deptName, e.empName
FROM departments AS d
CROSS JOIN employees AS e
ORDER BY d.deptID, e.empID
LIMIT 8;
```

deptName	empName
School of Engineering and Technology	Ananya Sharma
School of Engineering and Technology	Rohan Mehta
School of Engineering and Technology	Priya Nair
School of Engineering and Technology	Vikram Singh
School of Engineering and Technology	Fatima Khan
School of Engineering and Technology	Arjun Patel
School of Engineering and Technology	Neha Gupta
School of Management	Ananya Sharma

```
SQL> SELECT empID, empName, deptName, salary
FROM employees
NATURAL JOIN departments;
```

empID	empName	deptName	salary
1	Ananya Sharma	School of Engineering and Technology	92000
2	Rohan Mehta	School of Management	78000
3	Priya Nair	School of Law	85000
4	Vikram Singh	School of Agricultural Sciences	76000
5	Fatima Khan	School of Engineering and Technology	88000
6	Arjun Patel	School of Medical Sciences	95000

```
SQL> SELECT e.empID, e.empName, d.deptName, e.salary
FROM employees AS e
INNER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptName	salary
1	Ananya Sharma	School of Engineering and Technology	92000
2	Rohan Mehta	School of Management	78000
3	Priya Nair	School of Law	85000
4	Vikram Singh	School of Agricultural Sciences	76000
5	Fatima Khan	School of Engineering and Technology	88000
6	Arjun Patel	School of Medical Sciences	95000

```
SQL> SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
FROM employees AS e
LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptID	deptName	salary
1	Ananya Sharma	1	School of Engineering and Technology	92000
2	Rohan Mehta	2	School of Management	78000
3	Priya Nair	3	School of Law	85000
4	Vikram Singh	4	School of Agricultural Sciences	76000
5	Fatima Khan	1	School of Engineering and Technology	88000
6	Arjun Patel	5	School of Medical Sciences	95000
7	Neha Gupta	99		54000

```
SQL> SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
FROM employees AS e
RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptID	deptName	salary
1	Ananya Sharma	1	School of Engineering and Technology	92000
2	Rohan Mehta	2	School of Management	78000
3	Priya Nair	3	School of Law	85000
4	Vikram Singh	4	School of Agricultural Sciences	76000
5	Fatima Khan	1	School of Engineering and Technology	88000
6	Arjun Patel	5	School of Medical Sciences	95000
		6	School of Humanities	

```
SQL> SELECT
```

```

        e.empName AS employee,
        m.empName AS manager
    FROM employees AS e
    LEFT JOIN employees AS m ON e.managerID = m.empID
    ORDER BY e.empID;
employee      manager
-----
Ananya Sharma
Rohan Mehta   Ananya Sharma
Priya Nair    Ananya Sharma
Vikram Singh  Rohan Mehta
Fatima Khan   Ananya Sharma
Arjun Patel   Priya Nair
Neha Gupta    Rohan Mehta

```

7. Results

CROSS JOIN	INNER / NATURAL	LEFT OUTER	RIGHT OUTER	SELF JOIN
42	6	7	7	7
6 depts × 7 emps	Matched deptID only	+ Neha (NULL dept)	+ Humanities	Employee → manager

Figure 6 — Result set sizes from this lab dataset

Join type	Rows returned	Unmatched handling
CROSS JOIN	42	No condition — every dept × every employee
NATURAL JOIN	6	Drops non-matching rows from both sides
INNER JOIN	6	Same as NATURAL JOIN here
LEFT OUTER JOIN	7	Keeps Neha Gupta (deptID 99); deptName is NULL
RIGHT OUTER JOIN	7	Keeps Humanities (deptID 6); employee cols are NULL
SELF JOIN	7	All employees; Ananya has NULL manager (top of chain)

CROSS JOIN produces the largest result because it ignores relationships entirely. INNER JOIN and NATURAL JOIN return only the six employees whose deptID exists in departments. Outer joins add the orphan employee (left) or orphan department (right). SELF JOIN does not change row count here because a LEFT JOIN keeps every employee and simply leaves the manager column null for Ananya Sharma, who has no manager.

8. Conclusion

SQL joins answer different questions about related data. CROSS JOIN asks “every combination?” INNER JOIN asks “only matches?” Outer joins ask “keep everyone from this side even without a match?” SELF JOIN asks “how do rows in this table relate to other rows in the same table?” Choosing the right join depends on whether you need all combinations, only matches, or orphan rows from a specific side.